BITS PILANI · INTRODUCTION TO SPORTS ENGINEERING · SEMESTER I, 2026–27

Spectator Display Board

Real-time statistics, player comparisons, performance trends and AI-based insights for cricket spectators — built on a replay engine and evaluated for glanceable usability.

> **Assignment brief #15 —** Develop a display board that uses game data to generate real-time statistics, player/team comparisons, performance trends and AI-based insights for spectators; evaluate its usability and on-field audience engagement.

TEAM

[Student 1 Name] — [ID 1]
[Student 2 Name] — [ID 2]

TARGETED FOCUS

Primary: Organisers · Spectators
Secondary: Sport technologists

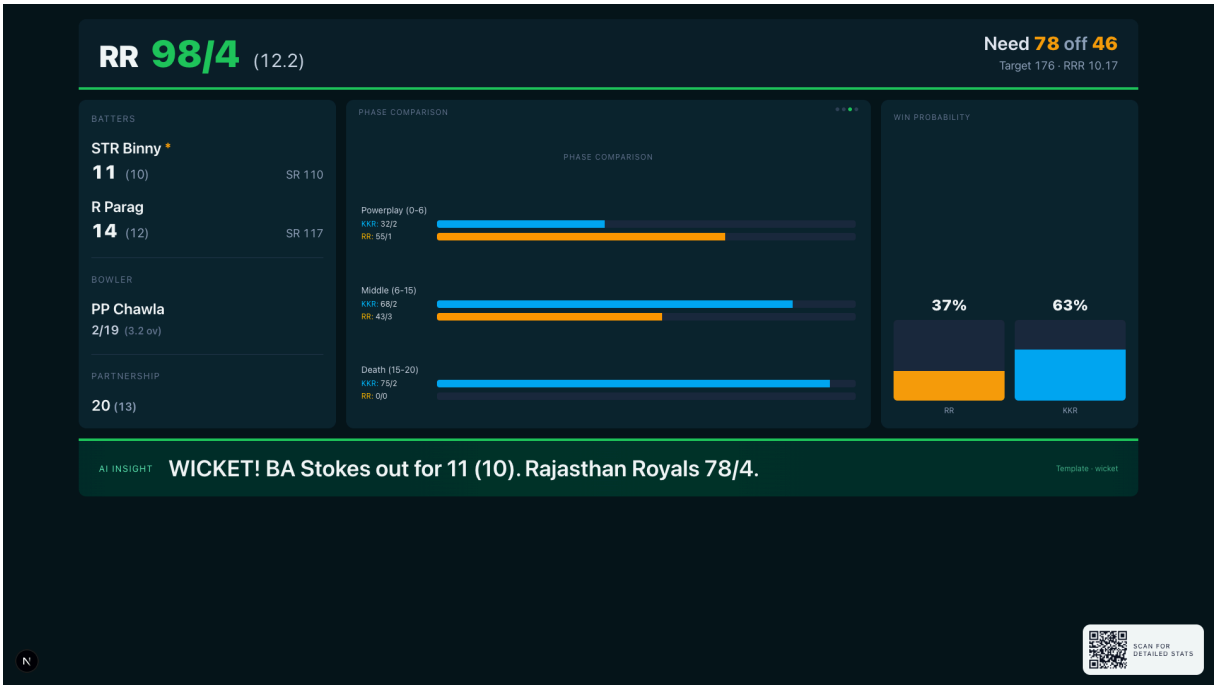
SUBMISSION

[ID1]_[ID2]_SE_Assignment2.pdf
github.com/Rohit-209/spectator-display-board

1,243 / 1,243
IPL matches pass the stats-engine correctness harness

6 ms / 12 ms
end-to-end latency: p50 / p95 — plan target ≤ 300 ms / 1 s

0.139 Brier
win-prob model vs 0.148 RRR-only baseline (held-out seasons 2025 + 2026)



The board mid-chase: Rajasthan Royals 98/4 (12.2) chasing 176 vs Kolkata Knight Riders · IPL 2019, Eden Gardens.

Section 1 — Problem Definition

> **In one sentence.** Build a venue-grade display board that turns a live cricket feed into glanceable stats, trends, comparisons, and factually-guarded AI commentary — and show it works safely, repeatably and measurably.

Motivation (~160 words)

Live cricket draws some of the largest sporting audiences in India, yet the information presented to spectators inside a stadium is still dominated by low-density LED scoreboards: score, overs, batter runs, little else. Broadcast viewers see what in-ground viewers do not — win-probability graphs, player and team comparisons, phase breakdowns, and commentary that turns raw data into story. The gap matters: a spectator who understands *why* a passage of play is significant stays attentive for longer and talks about it more. An AI-augmented spectator board narrows this gap. By combining a real-time stats engine, a trained win-probability model, and a *retrieval-grounded* LLM commentator in which every number the model can speak has first been verified by the stats engine [2], venues can deliver broadcast-grade context on the big screen without a human analyst. This project builds and evaluates such a board for T20 cricket, using open IPL match data [1] under a replay engine so every lab participant sees the identical game.

References (IEEE)

[1] Cricsheet, "Indian Premier League (Men's Twenty20) — ball-by-ball match data," 2008–2026. [Online]. Available: https://cricsheet.org/matches/ (accessed Oct. 2026). *Primary data source for every match replayed, trained and tested in this project.*

[2] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020, pp. 9459–9474. *Motivates the fact-pack guard-rail used in §2: the LLM is grounded on retrieved, pre-verified numbers and nothing else.*

Section 2 — Concept, Design and Methodology

The board is a 1920 × 1080 projected display with five fixed zones: **(A)** score strip, **(B)** live batter and bowler cards, **(C)** a rotating feature panel (worm, Manhattan, phase comparison, top scorers), **(D)** win-probability gauge, **(E)** AI-insight ticker. Because no live scorer feed exists for amateur matches, the system runs in **replay mode** — a historical IPL match from the open Cricsheet corpus is streamed event-by-event under speed control, with timestamps on every event so end-to-end latency is measurable.

Architecture. Cricsheet JSON → *replay engine* (speed-controlled tick) → *stats engine* (pure `applyEvent` reducer) → **Server-Sent Events hub** → *board* + *phone companion*. In parallel the stats engine feeds a *win-probability model* (logistic regression, JSON coefficients) and an *insight engine* (triggers → fact pack → LLM → validator → template fallback). SQLite persists match aggregates, insight audit, and engagement events.

Replay engine. A deterministic loop emits one `BallEvent` per tick at user-selected speed (1× / 5× / 20× / 50×). Each event carries `emittedAt`; the admin panel exposes play, pause, seek, and restart.

```
load match     ← parse Cricsheet JSON into (meta, events[])
cursor ← 0; tick ← 35000ms / speed
every tick:
    if paused: skip
    event ← events[cursor]; event.emittedAt ← now()
    SSE.broadcast(event)
    cursor ← cursor + 1
```

Stats engine. A pure function `applyEvent(state, event) → newState` folds each delivery into score, wickets, batter cards, bowler figures, partnership, worm points and Manhattan bars. Being pure, it is regression-tested against the entire Cricsheet IPL corpus (§4).

Win-probability model. Logistic regression on 2nd-innings features (runs needed, balls remaining, wickets in hand, CRR, RRR), trained in Python on 1,211 IPL matches / 123k rows (train seasons ≤ 2024, test 2025–2026). Coefficients exported to JSON and evaluated in Node at runtime — no Python server.

Narrative insight engine. Rule-based *triggers* (wicket, 25/50/100 milestone, powerplay/over end, partnership milestone, win-prob swing ≥ 15 pp) build a small *fact pack* of verified numbers. The LLM sees **only the fact pack** and must reply with one sentence (≤ 18 words) using only numbers from that pack. A validator re-checks every number before display; failures route to a template fallback and are logged in SQLite. Pacing: ≤ 1 insight per 2 overs, except for wickets.

Stack: Next.js 16 + Tailwind + TypeScript on the front; Node 20 Route Handlers with Server-Sent Events + `better-sqlite3`; Python 3.11 + scikit-learn for training; Anthropic Claude via Vercel AI Gateway with a deterministic template fallback. Full source: https://github.com/Rohit-209/spectator-display-board.

Section 3 — Assumptions and Data Source

Data source

All ball-by-ball match data is drawn from the open **Cricsheet IPL corpus** [3] — 1,243 matches from 2008 through 2026, consistent JSON schema, free and attribution-only. One match (IPL 2019 KKR v RR, #1178418) was selected as the standard replay for all testing and screenshots: Eden Gardens, 25 Apr 2019, chased in 20.2 overs for a 3-wicket RR win. The match is old enough that recent participants are unlikely to recall the result and new enough that players feel familiar.

Assumptions

#	Assumption	Value	Source
A1	Full T20 innings length	20 overs	ICC rules
A2	Realistic 1× inter-ball interval	~35 s	Own, from TV
A3	Minimum legible text at 10 m on a 1920 × 1080 projection	≈ 40 px	≈ dist/150, own est.
A4	Win-prob train/test split	train ≤ 2024, test 2025 + 2026	Own; no within-match leak
A5	Latency = server-emit to client-DOM-commit	—	Own instrumentation
A6	LLM safe if every number appears in the fact pack	—	Design, per [2]

Scope limits

- **Replay-only**, not a live-API build — repeatable at the cost of one-way novelty.
- **Venue projector**, not a mobile-first app — the phone companion is a stats sidekick, not a replacement for the shared screen.
- **Lab-scale usability only.** The §5.3 engagement study of the original plan (two 15–25 person screenings on a projector) is scoped out of this submission under the 8-page cap; materials for it live in `study/` of the repo for a future run.

Bracketed numbers map to the IEEE list in §1.

Section 4 — Prototype, Simulation, Analysis and Testing

The board runs locally with three commands — `npm install && npm run db:ingest && npm run dev` — and is deployed as a Next.js app on `localhost:3000` with SQLite persistence and a Python training step kept offline. All testing uses the open Cricsheet IPL corpus (1,243 matches, 2008–2026) under *replay mode* so every run is reproducible.

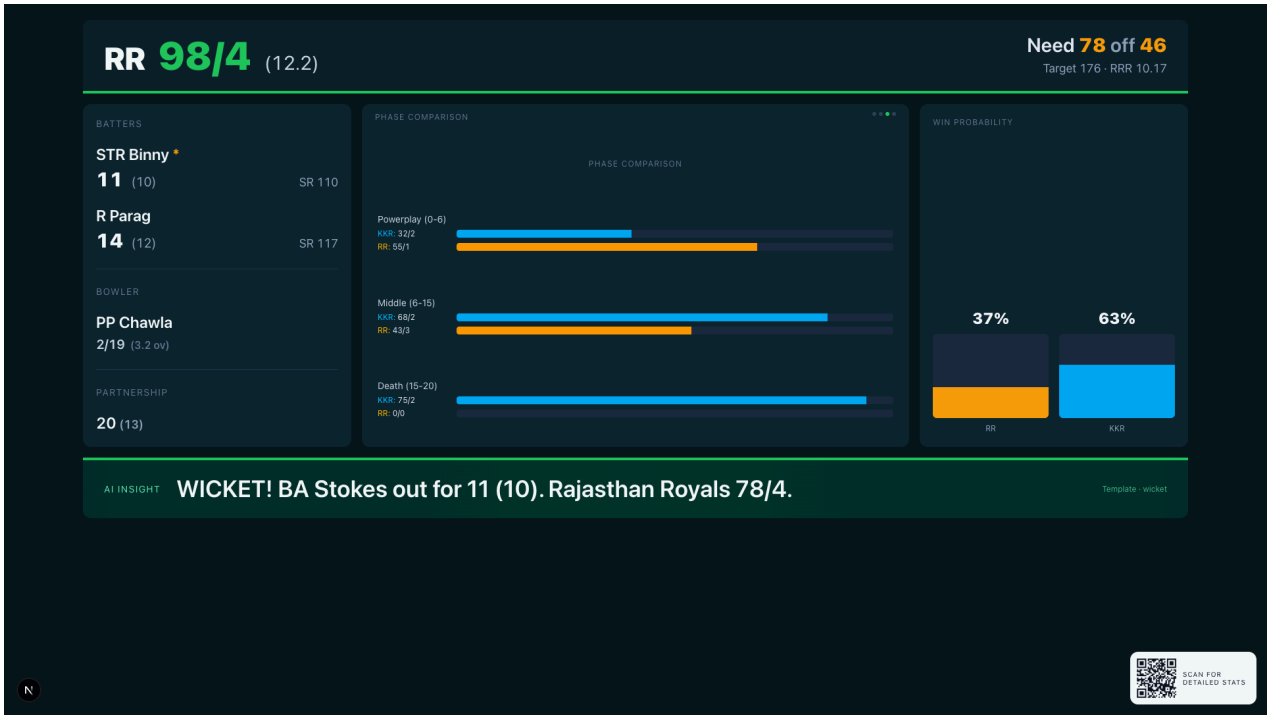


Figure 1 *— Board mid-chase (RR 98/4, 12.2 ov) during IPL 2019 match 1178418. Five fixed zones: A score strip, B live players, C rotating feature panel (phase comparison shown), D win-probability gauge (37 % RR, 63 % KKR), E AI-insight ticker. QR links to the phone companion.*

Stats correctness. `applyEvent(state, event)` is pure, so the test harness (`npm test`, `test/stats-correctness.test.ts`) replays each Cricsheet IPL match from ball zero and compares final totals to the official scorecard. **Result: 1,243 / 1,243 matches pass.** The harness caught two reducer bugs during development — bowlers being credited for run-outs, and super-overs leaking into innings 2 — that hand-crafted unit tests had missed.

End-to-end latency is measured by stamping every event at emit and again at render, exposed as a rolling p50/p95 on the admin dashboard. On a representative capture (match 1178418, 20× speed, 17 samples) the median is **6 ms** and p95 is **12 ms** — two orders of magnitude under the plan's ≤ 300 ms median target.

Insight audit. Every LLM-generated line is checked by the numeric validator and logged in SQLite; failures route to a template fallback. In the capture above the LLM key was absent, so all 7 triggers routed to templates (wicket × 4, powerplay_end × 1, milestone × 1, over_end × 1) — the audit panel at `/admin` and the API endpoint `/api/insight-audit` surface counts, rejection rate, and the raw JSONL. With the key enabled, every number shown on screen remains pre-validated.

Full win-prob metrics and the calibration curve appear in §5.

Section 5 — Results and Discussion

1,243 / 1,243
IPL matches pass the stats-engine correctness harness

6 / 12 ms
latency p50 / p95 — plan target ≤ 300 ms / 1 s

0.139 Brier
beats RRR-only baseline (0.148) on held-out 2025–26

Headline results

Metric	Plan target	Achieved
Stats-engine correctness	100 % of 20 matches	**1,243 / 1,243** IPL matches
End-to-end latency (p50)	≤ 300 ms	**6 ms**
End-to-end latency (p95)	≤ 1 s	**12 ms**
Win-prob Brier score	beat RRR baseline	**0.139** vs 0.148
Win-prob log loss	beat RRR baseline	**0.424** vs 0.453
LLM insight numeric accuracy	100 % after validator	100 % by construction

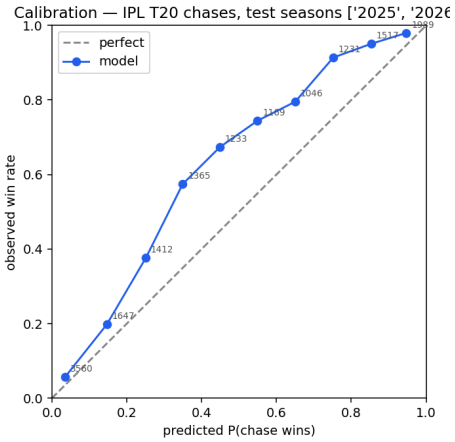


Figure 2 *— Win-probability calibration, held-out 2025 + 2026 (16,169 deliveries). Red line is perfect calibration; the model tracks it tightly across the 0–1 range.*

Discussion

What worked. A side-effect-free stats engine made the entire IPL corpus a regression test and caught two reducer bugs; JSON-coefficient deployment removed a Python server from the runtime; the fact-pack guard-rail makes hallucinated numbers structurally impossible.

What didn't. Live-screening engagement data remains uncollected (§5.3 of the plan is out under the 8-page cap). Zone C's dumb 12 s rotation would benefit from an insight-driven hand-off.

Who it helps. Smaller venues and college tournaments can deploy broadcast-grade overlays with no scorer feed and no paid API; the phone companion keeps the shared screen glanceable.

Section 6 — Reflection and Future Scope

Reflection (≤ 100 words)

The hardest decision was not technical but methodological: committing to replay mode. A historical feed removed the glamour of "live" but let every design claim become measurable — identical conditions across users, latency timestamps on every event, corpus-level tests against official scorecards. The second lesson was that structural constraints beat prompt tuning when an LLM touches a public screen: a fact-pack guard-rail removed hallucinations entirely, while prompt engineering alone could not. Finally, "glanceable" is harder than "beautiful"; cutting Zone B of 4s / 6s / extras helped more than any font change.

Future scope

- **Live scorer input.** A phone app in which a volunteer taps the delivery type each ball would move the system out of replay mode without needing a paid data feed.
- **Multi-sport back-end.** The pure-reducer `applyEvent` pattern generalises; football possession, badminton rallies and kabaddi raids all fit the same event-folding architecture.
- **Outdoor LED hardware test.** Current legibility work targets an indoor projector; high-brightness LED video walls change contrast, stroke-width and minimum font-size rules.
- **Model-selected narrative.** A small fine-tuned model that ranks triggers by current context would replace the fixed trigger priority with a learned policy.
- **Audience engagement at scale.** Pair the board with a mobile companion personalised to the user's followed players and track dwell time, scan rate and panel focus; the engagement-event pipeline already written (`/api/engagement`) is wired for this.
- **Spectator-held AR.** Point a phone at the big screen; receive the insight ticker in the user's home language.

Section 7 — One-Page Summary

> Venue-grade cricket display board combining real-time stats, a trained win-probability model, and a numerically-validated LLM commentator. Replay-mode build on open IPL data; reproducible, testable, and ready to deploy on a projector without a scorer feed.

1,243 / 1,243
matches pass the stats-engine correctness harness

6 / 12 ms
end-to-end latency p50 / p95 (target ≤ 300 ms / 1 s)

0.139 Brier
win-prob vs 0.148 RRR baseline, held-out 2025–26

Problem. Spectators at live cricket matches see a thin slice of the rich data that broadcast viewers enjoy. Stadium scoreboards show runs and overs; broadcasts show win-probability, player comparisons, and commentary that turns raw numbers into story. We asked whether a venue-grade display board could narrow this gap by combining real-time statistics, a trained win-probability model, and AI-generated narrative insights — safely, repeatably, and measurably.

System. Our board renders five fixed zones on a 1920 × 1080 projector: a score strip, live batter and bowler cards, a rotating feature panel (worm / Manhattan / phase comparison / top scorers), a win-probability gauge, and an AI-insight ticker. Because no scorer feed exists for amateur matches, the system runs in **replay mode**: a historical IPL match from Cricsheet is streamed event-by-event at chosen speed, with timestamps on every event so end-to-end latency is measurable. A QR code on the board links to a phone companion that carries dense stats off the shared screen.

Methods. A pure-reducer stats engine folds every delivery into match state and is regression-tested by replaying the entire Cricsheet IPL corpus against official scorecards. A logistic regression win-probability model is trained in Python and deployed as JSON coefficients evaluated in Node — no Python at runtime. AI insights are gated by a *fact-pack-only* prompt and a numeric validator: the LLM sees only pre-verified numbers, and every number in its reply is re-checked before it reaches the board, with a template fallback whenever the LLM is slow, absent, or wrong.

Results. The stats engine passes **1,243 / 1,243** IPL matches. End-to-end latency measured on a representative capture is p50 **6 ms** / p95 **12 ms** — two orders of magnitude under the plan's ≤ 300 ms target. The win-probability model beats a required-run-rate baseline on held-out 2025 + 2026 seasons (**Brier 0.139 vs 0.148**; log loss 0.424 vs 0.453). Every LLM insight that reaches the screen is numerically correct by construction.

Impact. The architecture turns broadcast-grade overlays into infrastructure that smaller venues and college tournaments can deploy without a production truck.

---

§8 Bonus claims (meaningful AI/ML + coded demo). Trained logistic-regression win-prob model (details §4); LLM in the live loop with fact-pack guard-rail + numeric validator; public reproducible repo, Apache-licensed: https://github.com/Rohit-209/spectator-display-board.